

QAOA Simulation Architecture Guide

Solving the Container Relocation Problem (CRP) via Quantum-Classical Hybrid Optimization

Author: Quantum Systems Architecture Group • Framework: Qiskit SDK • Document Type: Technical Reference & System Blueprint

1. EXECUTIVE SUMMARY & PROBLEM FRAMING

The **Container Relocation Problem (CRP)**—frequently encountered in container terminal logistics—is an NP-hard combinatorial optimization challenge. When a specific container situated beneath a stack must be retrieved for outbound transport, all blocking containers residing above it must be relocated to other stacks. The objective is to determine a valid sequence of relocation moves that minimizes the total count of secondary movements while obeying physical capacity and stacking constraints.

INITIAL STACK LAYOUT	TARGET RETRIEVAL STAGE
[Container 3] <-- Blocking	Relocate [3] -> Stack 2
[Container 2] <-- Blocking	Relocate [2] -> Stack 3
[Container 1] <-- Target	Retrieve [1]
-----	-----
Stack 1	Stack 1

To solve CRP on near-term quantum hardware, we employ the **Quantum Approximate Optimization Algorithm (QAOA)**. This approach converts the classical discrete decision space into an energy landscape, using a hybrid classical-quantum loop to find the ground state (minimum energy) corresponding to the optimal relocation schedule.

2. FUNDAMENTAL QUANTUM MECHANICS BEHIND QAOA

QAOA operates at the intersection of classical optimization and physical quantum dynamics. Four primary principles govern its execution:

2.1 Superposition

Instead of evaluating candidate relocation sequences sequentially, an n -qubit system is initialized in an equal superposition of all 2^n basis states:

$$|\psi_0\rangle = H^{\otimes n} |0\rangle^{\otimes n} = \frac{1}{\sqrt{2^n}} \sum_{z \in \{0,1\}^n} |z\rangle$$

2.2 Unitary Time Evolution

Quantum states evolve deterministically under a Hamiltonian operator according to the time-dependent Schrödinger equation:

$$i \hbar \frac{d}{dt} |\psi(t)\rangle = H |\psi(t)\rangle \implies |\psi(t)\rangle = e^{-i H t / \hbar} |\psi(0)\rangle$$

In QAOA, we construct unitary operators $U(H, \tau) = e^{-i \tau H}$ where τ acts as a continuous variable duration parameter adjusted by the classical optimizer.

2.3 Quantum Interference

By alternating between problem-specific phase shifts and state-mixing operations, destructive interference decreases the probability amplitudes of invalid or high-cost relocation sequences, while constructive interference amplifies the probability amplitudes of optimal solutions.

2.4 Quantum Measurement & Collapse

Upon measuring the state in the computational basis (Z -basis), the quantum state collapses to a classical bitstring $z \in \{0,1\}^n$ with probability $P(z) = |\langle z | \psi(y, \beta) \rangle|^2$. Repeated sampling generates a probability distribution biased toward optimal outcomes.

3. MATHEMATICAL FORMULATION: CONVERTING CRP TO QUBO

To map CRP onto quantum hardware, we translate physical constraints into a **Quadratic Unconstrained Binary Optimization (QUBO)** model.

3.1 Decision Variables

Let discrete time steps $t \in \{1, \dots, T\}$, stacks $s \in \{1, \dots, S\}$, tiers $p \in \{1, \dots, P\}$, and containers $c \in \{1, \dots, C\}$ define the domain:

- $x_{c,s,p,t} \in \{0, 1\}$: Equals 1 if container c resides at stack s , tier p , at time t ; otherwise 0.
- $y_{c,s,p,t} \in \{0, 1\}$: Equals 1 if container c is relocated to position (s,p) during step t ; otherwise 0.

3.2 Objective Function

Minimize total secondary moves:

$$C_{\text{obj}}(y) = \sum_{c,s,p,t} y_{c,s,p,t}$$

3.3 Constraint Penalty Functions

Constraints are added as quadratic penalty terms weighted by penalty multipliers $\lambda_1, \lambda_2, \lambda_3 \gg 0$:

1. **Unique Position Constraint:** Each container c must occupy exactly one position at any step t :

$$P_{\text{loc}} = \lambda_1 \sum_{c,t} (\sum_{s,p} x_{c,s,p,t} - 1)^2$$

2. **Occupancy Limit Constraint:** No coordinate (s,p) can hold more than one container simultaneously:

$$P_{\text{cap}} = \lambda_2 \sum_{s,p,t} (\sum_c x_{c,s,p,t}) (\sum_c x_{c,s,p,t} - 1)$$

3. **Gravity/Stacking Constraint:** A container cannot exist at tier $p > 1$ unless a supporting container occupies tier $p-1$:

$$P_{\text{grav}} = \lambda_3 \sum_{c,s,p>1,t} x_{c,s,p,t} (1 - \sum_{c'} x_{c',s,p-1,t})$$

3.4 Unifying into the Final QUBO Matrix

Combining terms yields a single upper-triangular scalar expression:

$$f(z) = z^T Q z = \sum_i Q_{ii} z_i + \sum_{i < j} Q_{ij} z_i z_j$$

4. PHYSICS MAPPING: THE ISING MODEL & COST HAMILTONIAN

To evaluate Q on qubits, we map binary variables $z_i \in \{0,1\}$ to the eigenvalues of the Pauli-Z operator $Z_i \in \{+1, -1\}$ using the transformation:

$$z_i \rightarrow (I - Z_i) / 2$$

Substituting this identity into $f(z)$ converts the classical cost function into an **Ising Spin Glass Hamiltonian** (H_C):

$$H_C = \sum_{i=1}^n h_i Z_i + \sum_{1 \leq i < j \leq n} J_{ij} Z_i Z_j + K \cdot I$$

Physical Insights:

- h_i (Local Bias): Represents effective magnetic field shifts derived from linear QUBO terms.
- J_{ij} (Coupling Strength): Represents two-qubit entanglement interaction strength derived from quadratic terms.
- K : Constant energy offset.

Key Principle: The ground state (lowest energy eigenvalue) of H_C corresponds exactly to the bitstring z^* that solves the Container Relocation Problem with minimum relocations.

5. THE QAOA QUANTUM CIRCUIT & PARAMETERIZED ANSATZ

QAOA constructs a parameterized quantum state $|\gamma, \beta\rangle$ by applying p alternating layers of two operators:

```
+-----+
|                                     |
|               QAOA CIRCUIT (p Layers)               |
|                                     |
|0> -H-|--- [ exp(-i γ_1 H_C) ] --- [ exp(-i β_1 H_B) ] --- ... --- Measure |
|0> -H-|--- [ exp(-i γ_1 H_C) ] --- [ exp(-i β_1 H_B) ] --- ... --- Measure |
|0> -H-|--- [ exp(-i γ_1 H_C) ] --- [ exp(-i β_1 H_B) ] --- ... --- Measure |
+-----+
```

5.1 Cost Operator Layer $U(H_C, \gamma_k)$

$$U(H_C, \gamma_k) = e^{-i \gamma_k H_C} = \prod_i e^{-i \gamma_k h_i Z_i} \prod_{i < j} e^{-i \gamma_k J_{ij} Z_i Z_j}$$

- Single-qubit terms $e^{-i \gamma h_i Z_i}$ map directly to $R_Z(2\gamma h_i)$ rotations.
- Two-qubit interaction terms $e^{-i \gamma J_{ij} Z_i Z_j}$ decompose into CNOT – $R_Z(2\gamma J_{ij})$ – CNOT gate sequences.

5.2 Mixer Operator Layer $U(H_B, \beta_k)$

$$H_B = \sum_{i=1}^n X_i \implies U(H_B, \beta_k) = e^{-i \beta_k H_B} = \prod_{i=1}^n e^{-i \beta_k X_i}$$

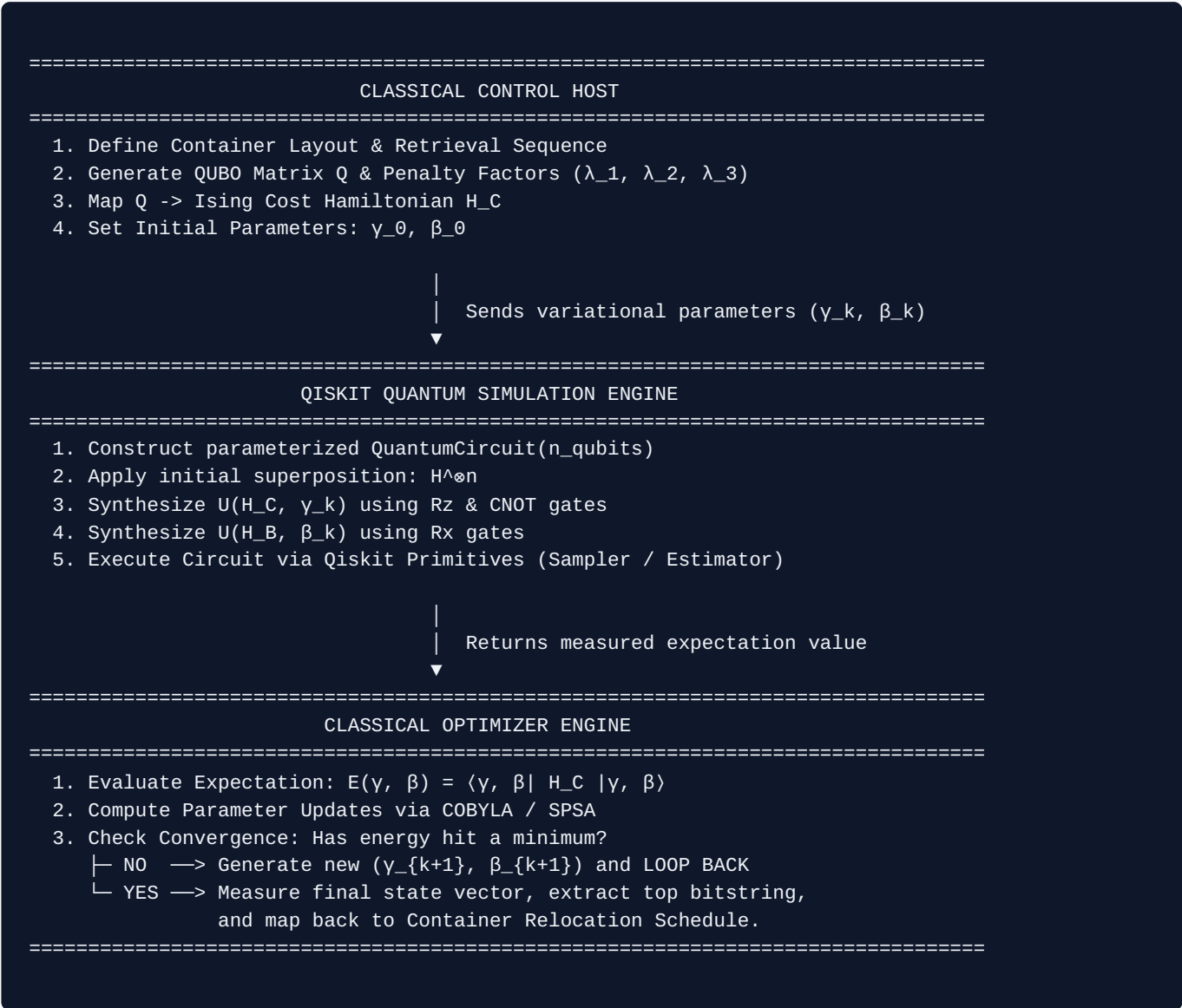
Implemented using single-qubit $R_X(2\beta_k)$ rotations across all qubits.

5.3 Complete Parameterized State Expression

$$|\gamma, \beta\rangle = [\prod_{k=1}^p e^{-i \beta_k H_B} e^{-i \gamma_k H_C}] H^{\otimes n} |0\rangle^{\otimes n}$$

6. SYSTEM ARCHITECTURE: HYBRID CONTROL LOOP

The QAOA pipeline alternates between classical parameters and quantum state evaluation in a closed-loop configuration:



7. END-TO-END SYSTEM OVERVIEW & EXECUTION PIPELINE

Below is the structured walkthrough of how the simulation engine is built, initialized, and executed within Qiskit:

PHASE A

Variable Mapping & Parameter Configuration

- **Input Data:** Specify container priority order, stack/tier limits (S, P), and max time steps (T).
- **Qubit Indexing:** Map each 4-tuple decision variable (c, s, p, t) to an integer qubit index $i \in \{0, \dots, n-1\}$.
- **Penalty Tuning:** Choose penalty multipliers $\lambda_1, \lambda_2, \lambda_3$ larger than the cost of a single relocation move.

PHASE B Automated QUBO & Hamiltonian Synthesis

- **Build Quadratic Program:** Populate Qiskit Optimization's QuadraticProgram with decision variables and quadratic penalty matrices.
- **Map to Ising Model:** Convert the model into an Ising spin Hamiltonian using ToIsing, producing a Qiskit SparsePauliOp representing H_C .

PHASE C Quantum Execution Loop (Qiskit Primitives)

- **Primitive Engine:** Instantiate Qiskit's Estimator primitive for energy evaluation (H_C), and Sampler for sampling bitstrings.
- **Ansatz Layers:** Set QAOA depth p (e.g. $p=1$ or $p=2$).
- **Classical Optimization:** Pass parameters to COBYLA/SPSA. Iteratively adjust (γ, β) until cost expectation stabilizes at a minimum.

PHASE D Result Extraction & Solution Decoding

- **Sampling:** Execute converged circuit with high shot counts (e.g., 8,192 shots).
- **Decoding:** Extract the most frequent bitstring z^* and convert 1s and 0s back to physical container coordinates $x_{c,s,p,t}$.
- **Verification:** Validate that the output sequence satisfies gravity and capacity rules, returning the optimal container relocation schedule.